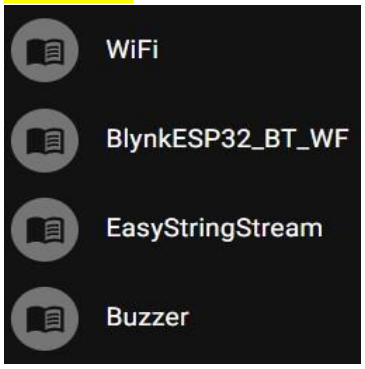
PRACTICAL NO : 01

PARKING SYSTEM

Components:

1. HC-SR04 Ultrasonic Distance Sensor (4)
2. ESP32
3. Buzzer

Libraries:



Connections:

1. ESP32:
    a. Left:
        i. 3V3 = VCC Pins of all UDS
        ii. 14 = TRIG_C
        iii. 13 = ECHO_C
    b. Right:
        i. GND.2 = GND Pins of all UDS, 1 (Buzzer)
        ii. 23 = 2 (Buzzer)
        iii. 19 = TRIG_A iv. 18 = TRIG_B
        iv. 5 = ECHO_D
        v. 17 = TRIG_D
        vi. 16 = ECHO_B
        vii. 2 = ECHO_A

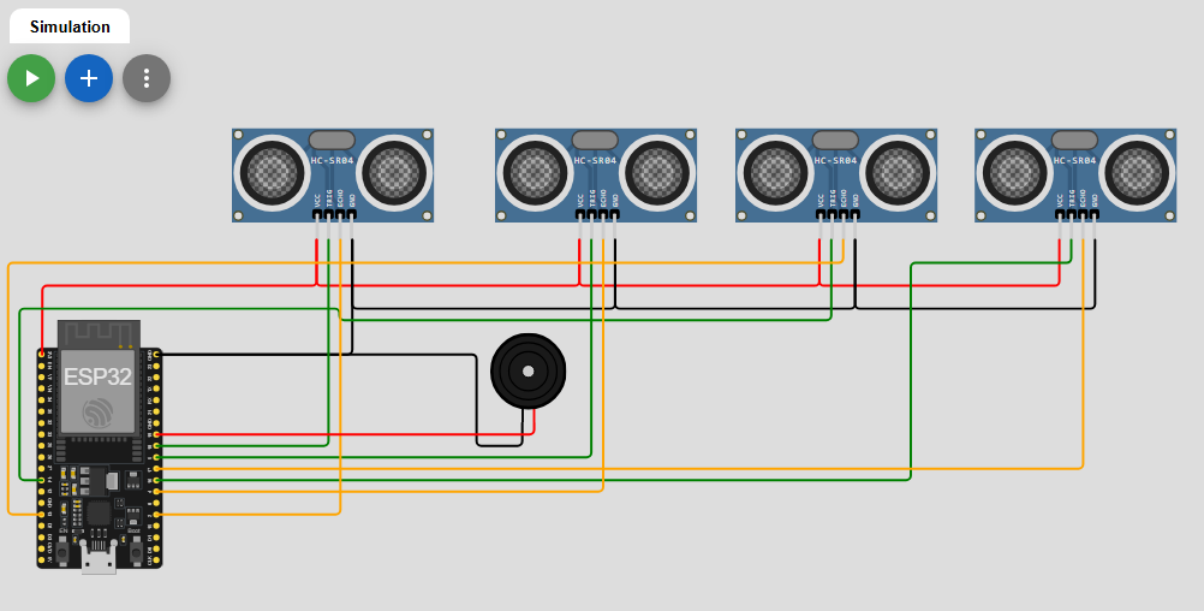
Code:

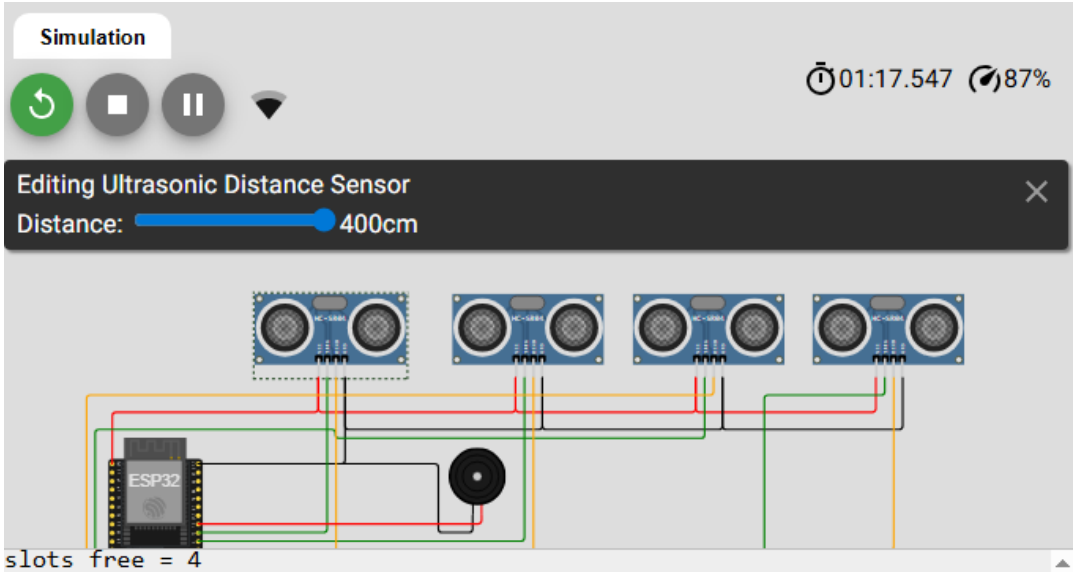
```
#define BLYNK_TEMPLATE_ID "TMPL3YauG93Qa"
#define BLYNK_TEMPLATE_NAME "Parking System"
#define BLYNK_AUTH_TOKEN "SWcZN-5k2YkOd9pIEhYi-d-ttPV-PnYz"


//including library files
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
```

```
//Defining wifi credentials
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Wokwi-GUEST";
char pass[] = "";

//Defining Pin numbers
#define ECHO_A 2
#define TRIG_A 18
#define ECHO_B 4
#define TRIG_B 5
#define ECHO_C 13
#define TRIG_C 14
#define ECHO_D 17
#define TRIG_D 16
#define buzzer 19

int a,b,c,d;
int Slot;

BlynkTimer timer;

void setup(){
  //PUT your setup code here, to run once
  Serial.begin(115200);

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  //Defining pin modes
  pinMode(TRIG_A, OUTPUT);
  pinMode(ECHO_A, INPUT);
  pinMode(TRIG_B, OUTPUT);
  pinMode(ECHO_B, INPUT);
  pinMode(TRIG_C, OUTPUT);
  pinMode(ECHO_C, INPUT);
  pinMode(TRIG_D, OUTPUT);
  pinMode(ECHO_D, INPUT);
  pinMode(buzzer, OUTPUT);

  Serial.begin(115200);
  Serial.println("Hello, ESP32!");
}

void loop() {
  //put your main code here, to run repeatedly
  a=check(TRIG_A,ECHO_A);
  tone(buzzer, 1000);
  noTone(buzzer);
  if(a==0) {
    Blynk.virtualWrite(V1, 0);
```

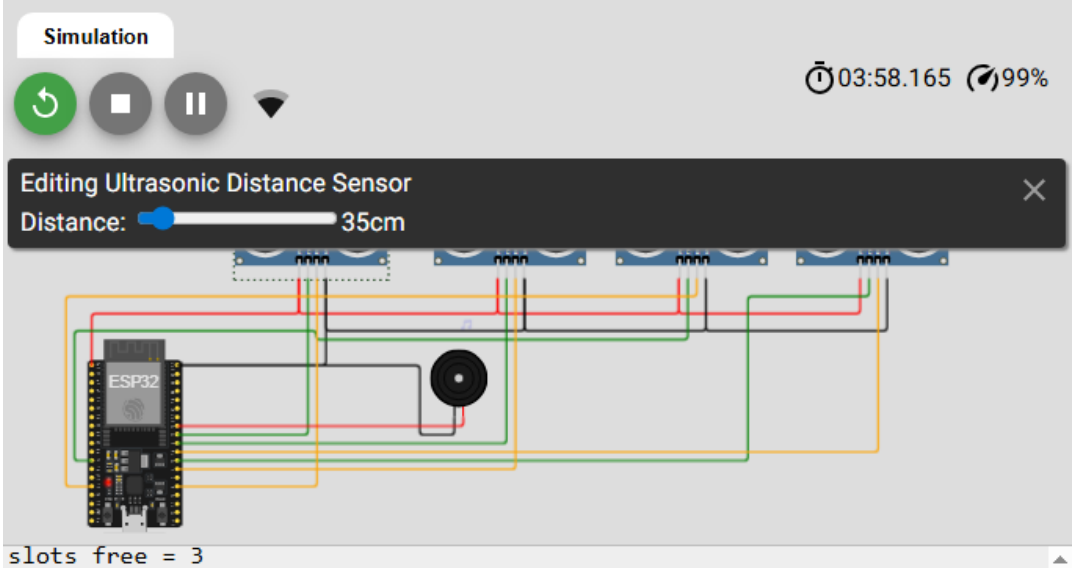
```
    tone(buzzer, 1000);
  }
  else {
    Blynk.virtualWrite(V1, 1);
    noTone(buzzer);
  }
  b=check(TRIG_B,ECHO_B);
  if(b==0) {
    Blynk.virtualWrite(V2, 0);
    tone(buzzer, 1000);
  }
   else {
    Blynk.virtualWrite(V2, 1);
    noTone(buzzer);
  }
  c=check(TRIG_C,ECHO_C);
  if(c==0) {
    Blynk.virtualWrite(V3, 0);
    tone(buzzer, 1000);
  }
   else {
    Blynk.virtualWrite(V3, 1);
    noTone(buzzer);
  }
  d=check(TRIG_D,ECHO_D);
  if(d==0) {
    Blynk.virtualWrite(V4, 0);
    tone(buzzer, 1000);
  }
   else {
    Blynk.virtualWrite(V4, 1);
    noTone(buzzer);
  }
  Serial.printf("%d %d %d %d \n",a,b,c,d);
  Slot=a+b+c+d;
  Serial.printf("slots free = %d \n",Slot);
  Blynk.virtualWrite(V0,Slot);
  delay(2000);
}

//function to check distance
float check(int trigerpin, int echopin){
  float distance, duration;
  digitalWrite(trigerpin, LOW);
  delay(2);
  digitalWrite(trigerpin, HIGH);
  delay(10);
  digitalWrite(trigerpin, LOW);
```

```
  duration=pulseIn(echopin, HIGH);
  distance=duration/2*0.034;
  Serial.print("Distance (in cms)=");
  Serial.println(distance);
  delay(10);
  if(distance>100){
    return 1;
  }
  else{
    return 0;
  }
}
```

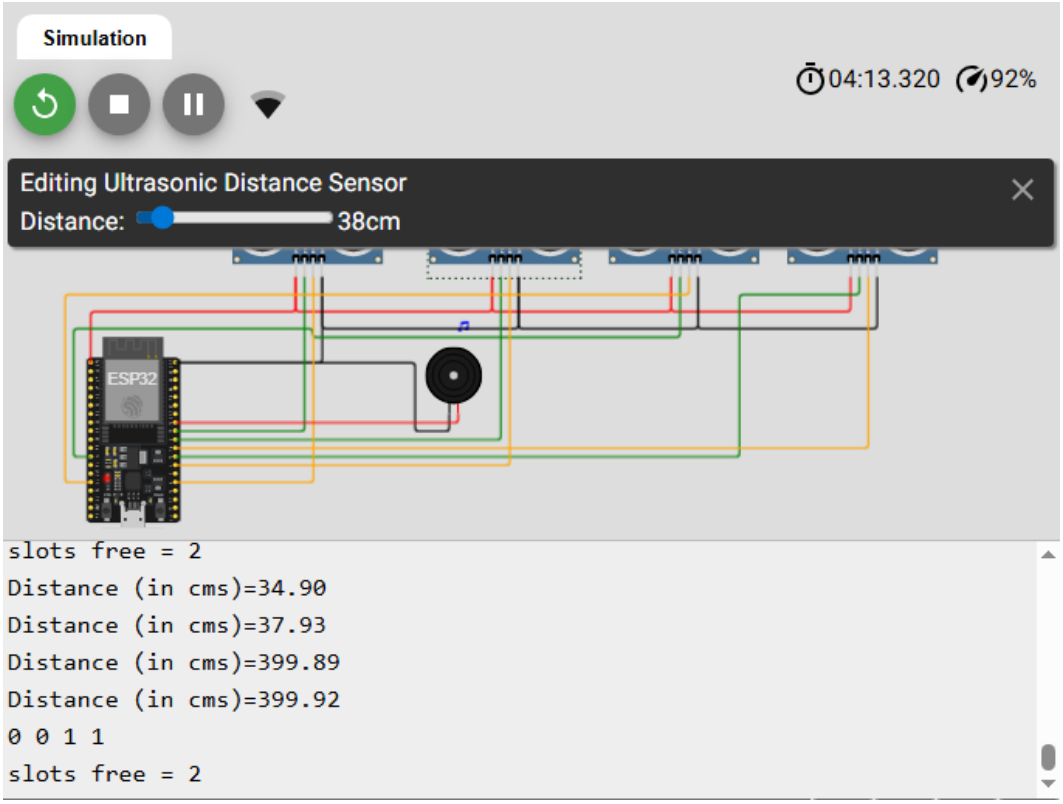


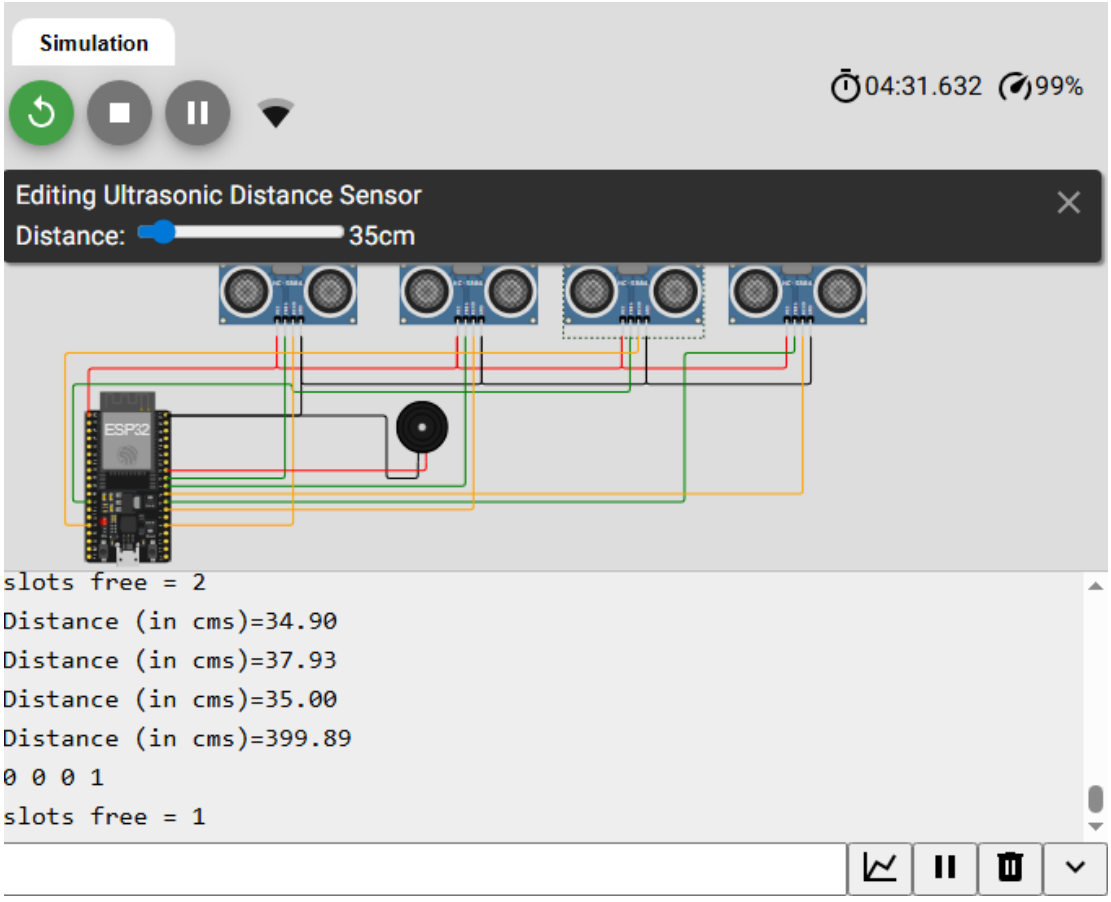


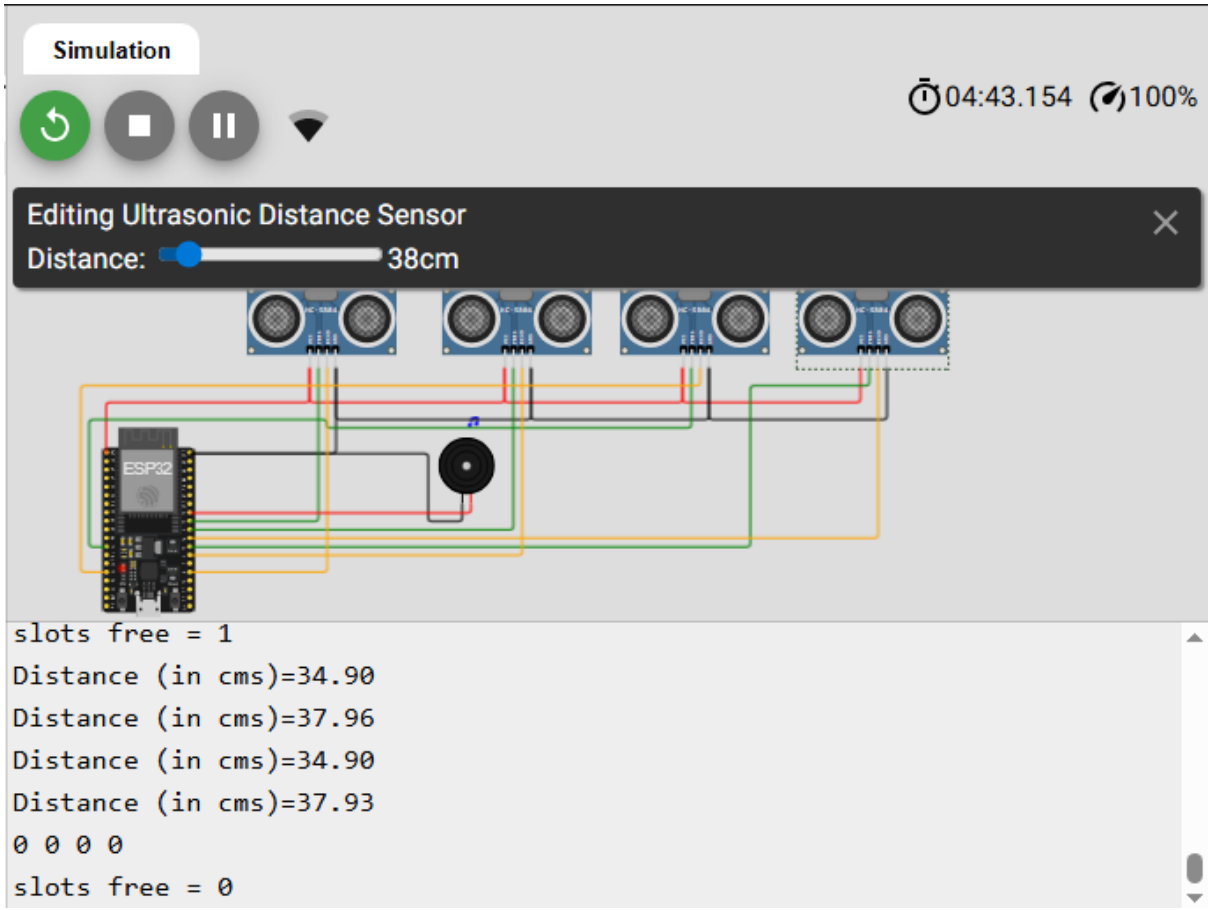

```
slots free = 4
Distance (in cms)=399.89
Distance (in cms)=399.89
Distance (in cms)=399.86
Distance (in cms)=399.89
1 1 1 1
slots free = 4
```



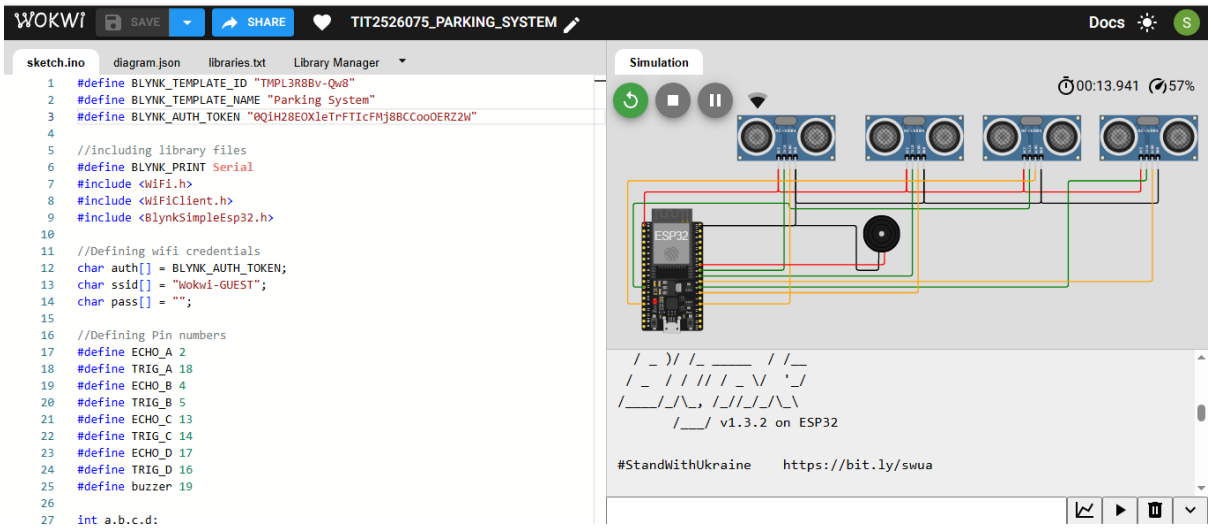

```
slots free = 3
Distance (in cms)=34.90
Distance (in cms)=399.89
Distance (in cms)=399.89
Distance (in cms)=399.91
0 1 1 1
slots free = 3
```


Simulation
04:13.320
92%
Editing Ultrasonic Distance Sensor
Distance:
38cm
ESP32
slots free = 2
Distance (in cms)=34.90
Distance (in cms)=37.93
Distance (in cms)=399.89
Distance (in cms)=399.92
0 0 1 1
slots free = 2


Simulation
04:31.632
99%
Editing Ultrasonic Distance Sensor
Distance:
35cm
ESP32
slots free = 2
Distance (in cms)=34.90
Distance (in cms)=37.93
Distance (in cms)=35.00
Distance (in cms)=399.89
0 0 0 1
slots free = 1



AFTER CONNECTING WITH BLYNK OUTPUT:



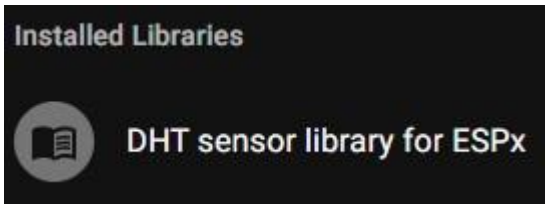
PRACTICAL NO : 02

ESP32 DHT22 Temp Humidity Sensor

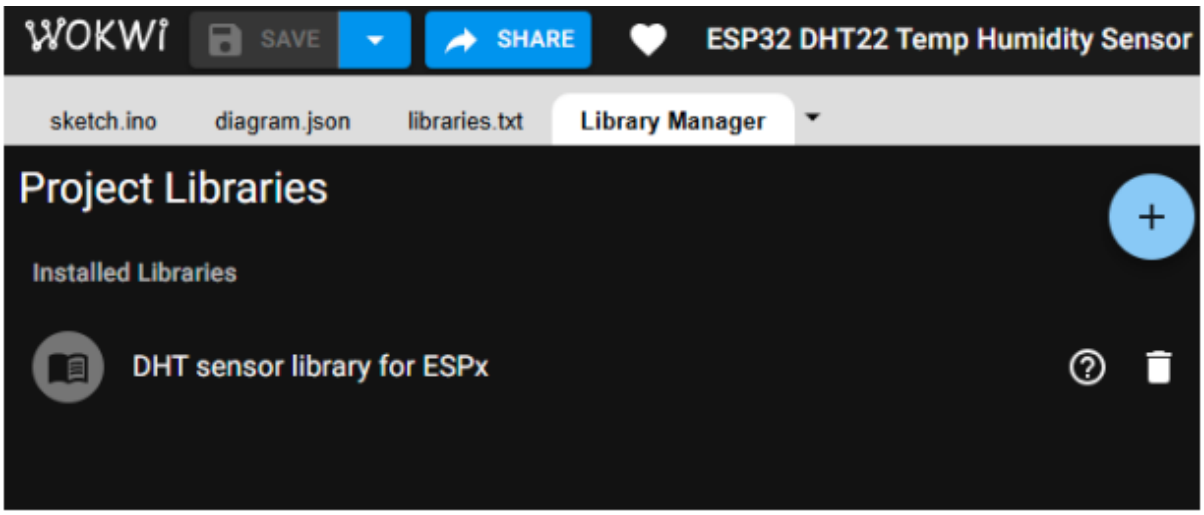
Connections:

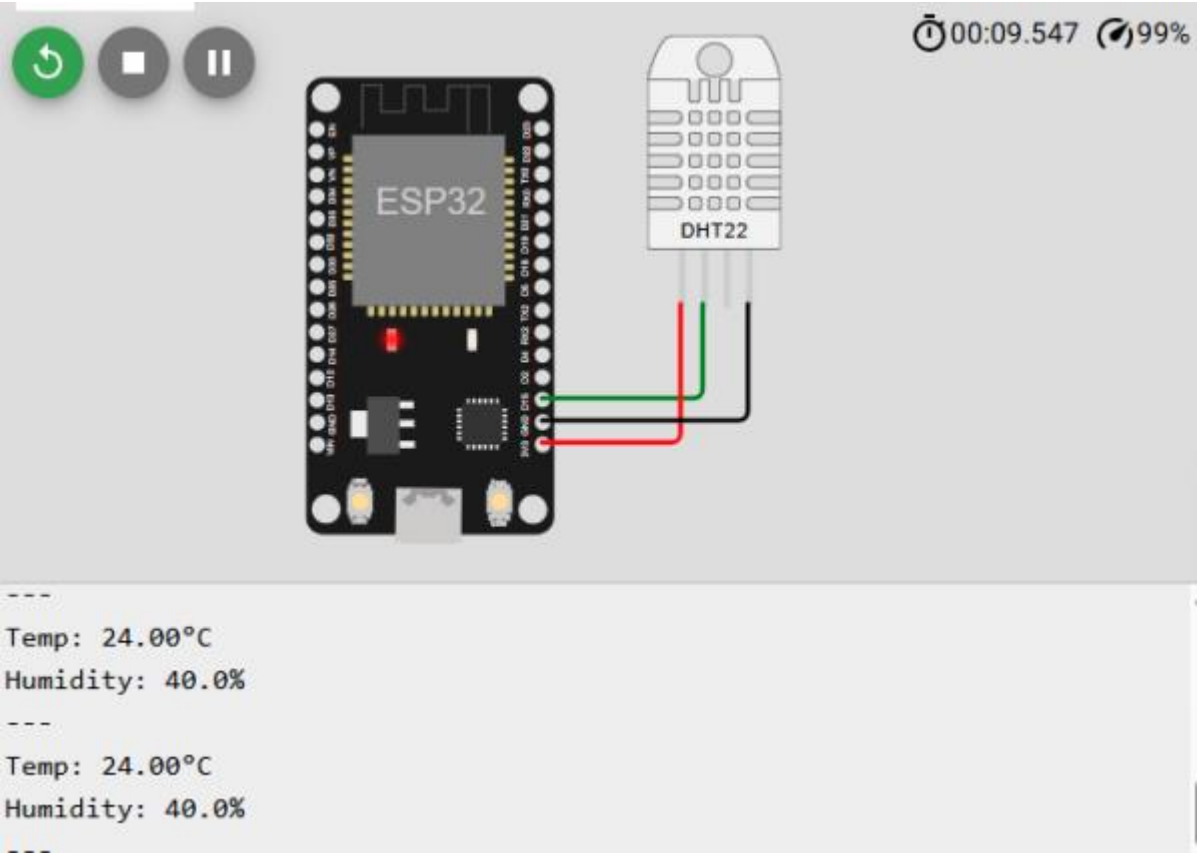
ESP32	DHT22
D15	SDA
GND1	GND
3V3	VCC

Libraries:




```
#include "DHTesp.h"
const int DHT_PIN = 15;
DHTesp dhtSensor;

void setup() {
  Serial.begin(115200);
  dhtSensor.setup(DHT_PIN,DHTesp::DHT22);
}

void loop()
{
  TempAndHumidity data = dhtSensor.getTempAndHumidity();
  Serial.println("Temp: " + String(data.temperature, 2) + "°C");
  Serial.println("Humidity: " + String(data.humidity,1) + "%");
  Serial.println("---");
  delay(2000);
}
```





PRACTICAL NO : 03

DISTANCE MONITORING SYSTEM

Components:

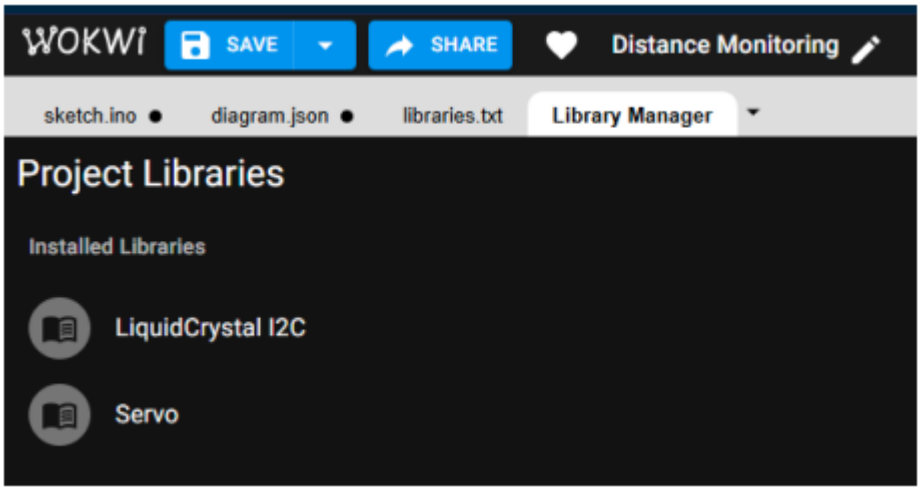
1. Arduino UNO
2. LCD 16X2 (I2C)
3. HC-SR04 Ultrasonic Distance Sensor
4. 4 LEDs
5. 4 Resistors (1 kΩ each) Connections of Ar:

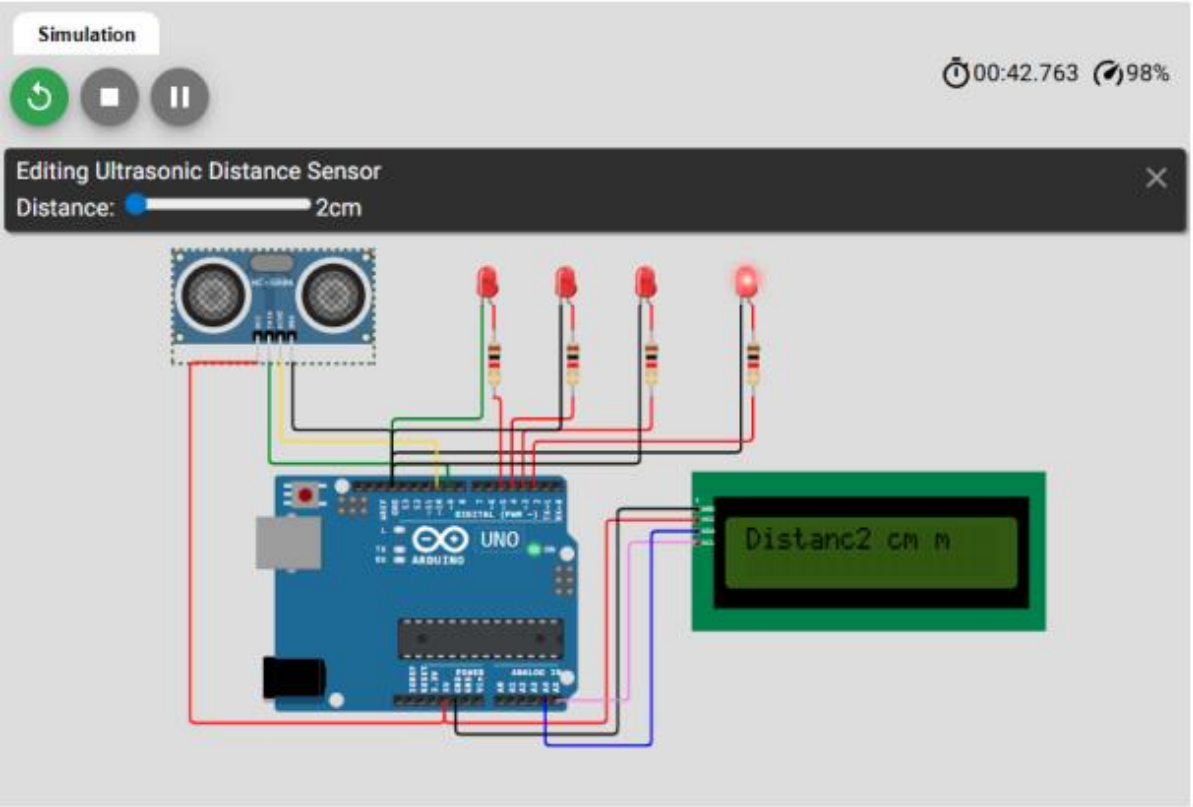
1. All A (Anodes) of the four LEDs connect to a resistor of 1 kΩ, (yellow band of the resistor facing downwards) the other end of the resistors connect to 2, 3, 4 and 5 pins of Arduino UNO respectively. All C (Cathodes) of the four LEDs are connected to GND.1 of Arduino UNO.
2. HC-SR04 Ultrasonic Distance Sensor:
   i. VCC – 5V
   ii. TRIG – 9
   iii. ECHO – 10
   iv. GND – GND.1 3.

3. LCD 16X2 (I2C):
   i. GND – GND.2
   ii. VCC – 5V
   iii. SDA – A4
   iv. SCL – A5

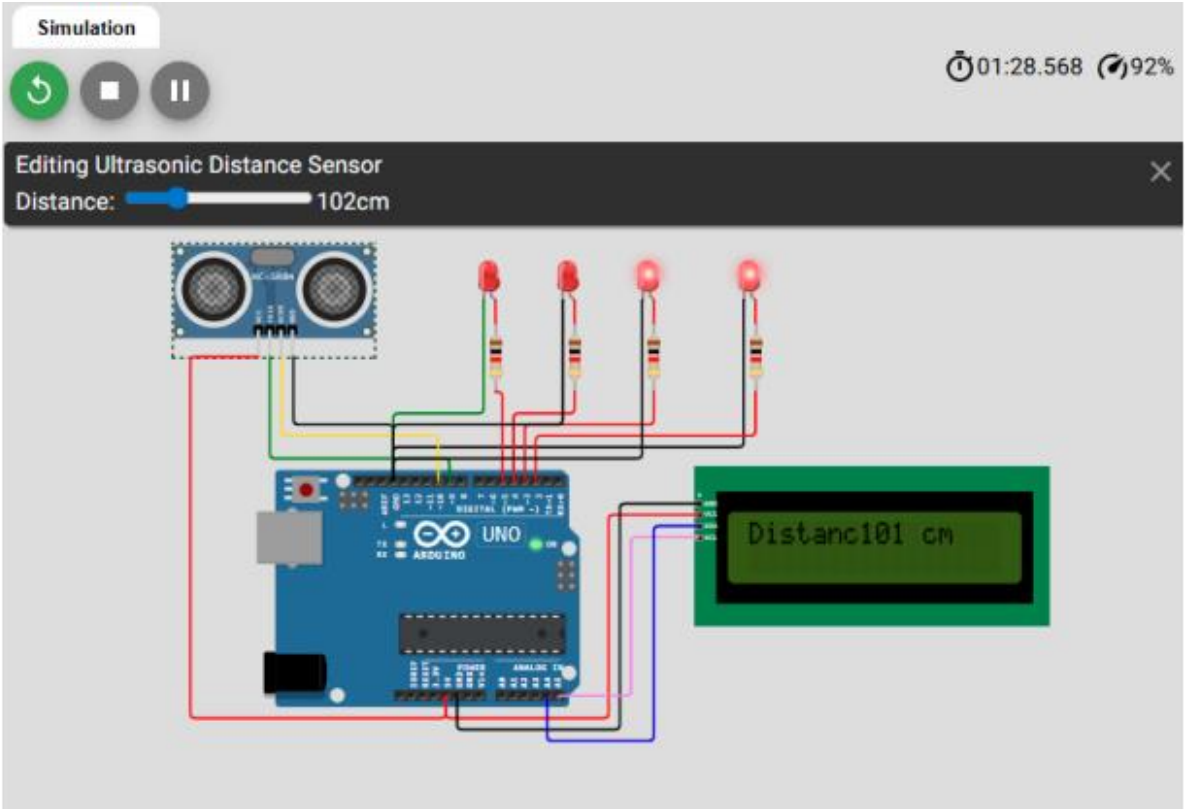
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

//Intialzing object LCD
LiquidCrystal_I2C lcd(0x27, 16, 2);

//Intialzing pin sensor ultrasonic
const int trigPin = 9;
const int echoPin = 10;

//Intialzing pin LED
const int led1 = 2;
const int led2 = 3;
const int led3 = 4;
const int led4 = 5;

void setup()
{
  //Intialize pin-mode
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(led3, OUTPUT);
  pinMode(led4,OUTPUT);

  //Intialize LCD
  lcd.begin(16,2);
  lcd.print("Distance ");
}

void loop()
{
  //sends the sound pulse ultrasonic
  digitalWrite(trigPin,LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin,HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin,LOW);

  //Hit the object and comes back the sound pulse
  long duration = pulseIn(echoPin,HIGH);
```

```
  //calculating the distance in Centimeters
  int distance = duration * 0.034 / 2;

  //Displaying the result on LCD
  lcd.setCursor(8,0);
  lcd.print("       "); //Clearing the previous characters
  lcd.setCursor(8,0);
  lcd.print(distance);
  lcd.print(" cm");

  //Activating (turning on) the LEDs based on the distance
  updateLEDs(distance);

  delay(500); //Delay to slow down the readings
}

void updateLEDs(int distance)
{
  if (distance < 50 ) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, LOW);
    digitalWrite(led3, LOW);
    digitalWrite(led4, LOW);
  } else if (distance >= 50 && distance < 150) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, LOW);
    digitalWrite(led4, LOW);
  } else if (distance >= 150 && distance < 250) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, HIGH);
    digitalWrite(led4, LOW);
  } else if (distance >= 250 && distance < 350) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, HIGH);
    digitalWrite(led4, HIGH);
  } else {
    digitalWrite(led1,LOW);
    digitalWrite(led2, LOW);
    digitalWrite(led3, LOW);
    digitalWrite(led4, LOW);
  }
}
```





Distance 2cm



Distance 102cm

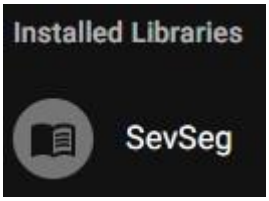
PRACTICAL NO : 04

SEVEN SEGEMENT:

Connections:

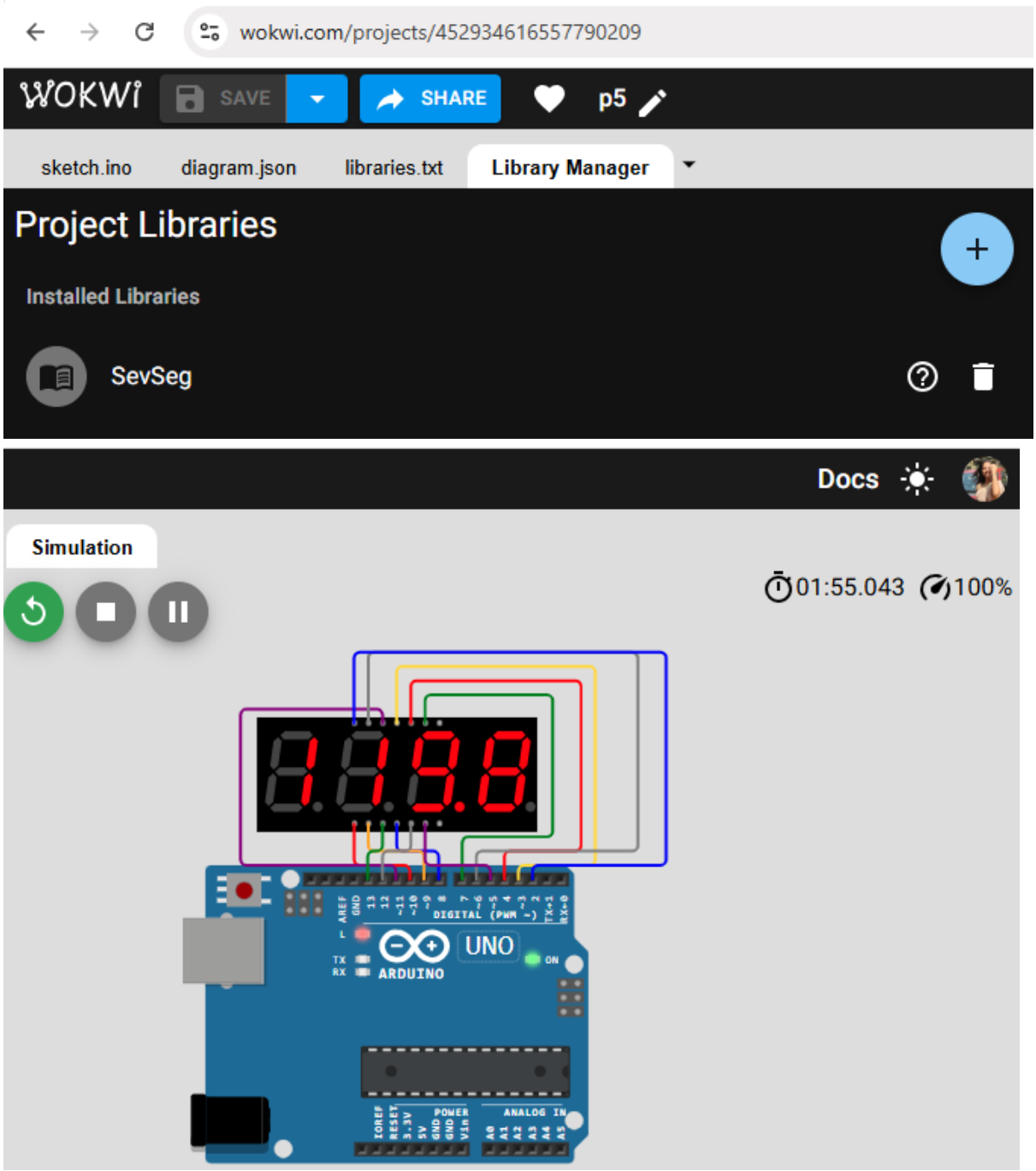
7 Seg Display	Arduino UNO
DIG1	2
A	6
F	11
DIG2	3
DIG3	4
B	7
E	10
D	9
DP	13
C	8
G	12
DIG4	5

Libraries:




```
#include "SevSeg.h"
SevSeg sevseg;

void setup() {
  // put your setup code here, to run once:
  byte numDigits=4;
  byte digitPins[]={2,3,4,5};
  byte segementPins[]={6,7,8,9,10,11,12,13};
  bool resistorsOnSegments=false;
  byte hardwareConfig=COMMON_ANODE;
  bool updateWithDelays=false;
  bool leadingZeros=false;
  bool disableDecPoint=false;

  sevseg.begin(hardwareConfig, numDigits, digitPins, segementPins,
   resistorsOnSegments, updateWithDelays, leadingZeros,disableDecPoint);
  sevseg.setBrightness(90);

}

void loop() {
  // put your main code here, to run repeatedly:
  static unsigned long timer = millis();
  static int deciSeconds = 0;

  if(millis() - timer >=100)
  {
    timer +=100;
    deciSeconds++;

    if(deciSeconds == 10000)
    {
      deciSeconds=0;
    }
    sevseg.setNumber(deciSeconds,1);
  }


   sevseg.refreshDisplay();
```

}


wokwi.com/projects/452934616557790209
WOKWI
SAVE
SHARE
p5
sketch.ino
diagram.json
libraries.txt
Library Manager
Project Libraries
Installed Libraries
SevSeg
Docs
Simulation
01:55.043
100%
UNO
ARDUINO

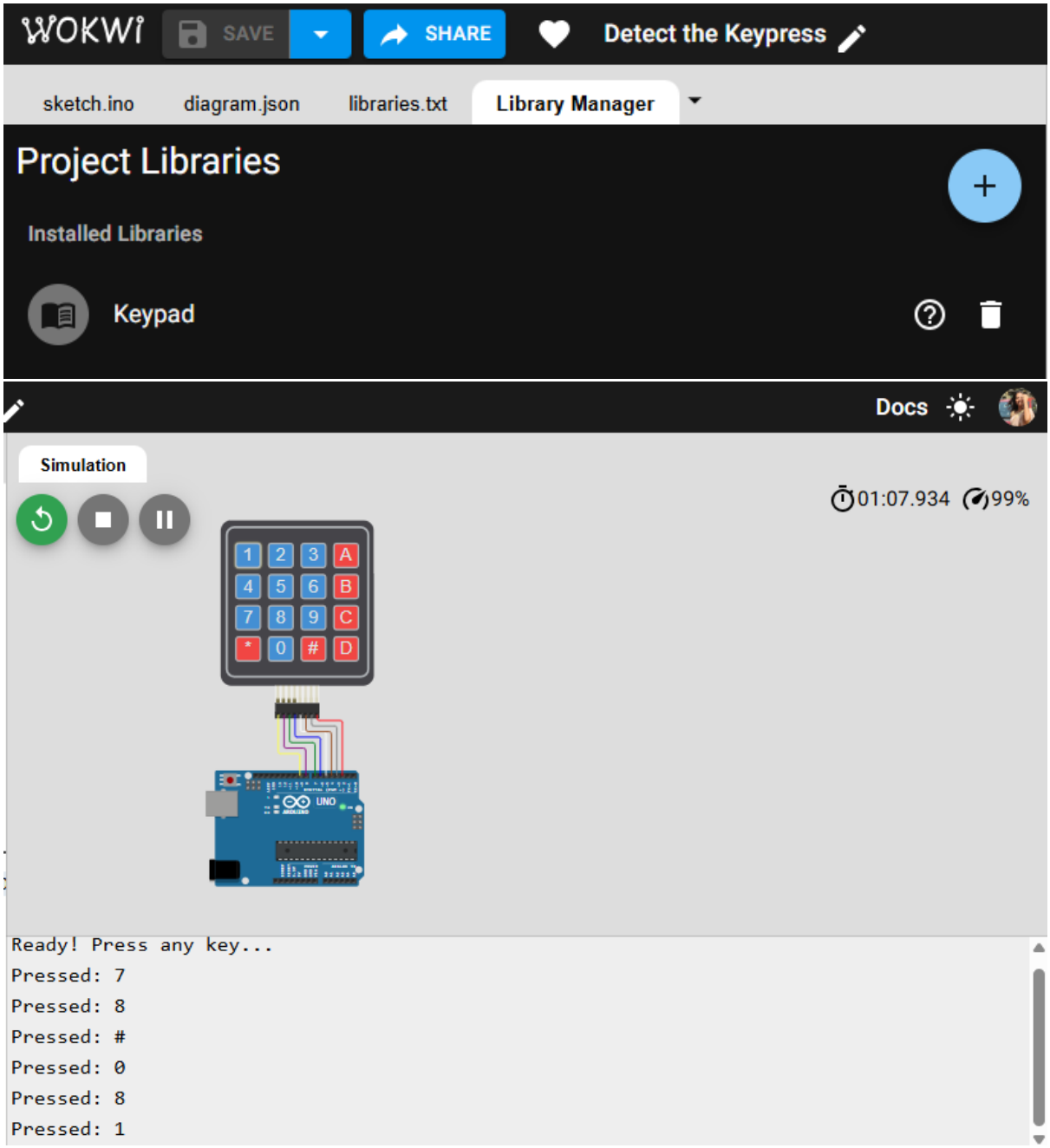
PRACTICAL NO : 05

DETECT THE KEYPRESS

Libraries

Components and connection

Arduino Pin	Connected To
D2	Column 1
D3	Column 2
D4	Column 3
D5	Column 4
D6	Row 1
D7	Row 2
D8	Row 3
D9	Row 4

```
char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

//Update these pin numbers to match your wiring in wokwi!
byte rowPins[ROWS] = {9,8,7,6};  //R1,R2,R3,R4
byte colPins[COLS] = {5,4,3,2};  //C1,C2,C3,C4

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void setup()
{
  Serial.begin(9600);
```

```
  Serial.println("Ready! Press any key... ");
}

void loop()
{
  char key = keypad.getKey();
  if(key)
  {
    Serial.print("Pressed: ");
    Serial.println(key);
  }
}
```